

AI-BDM

AI-Powered B2B Lead Generation Platform

Abdul Basit & Afrazia Umer

Complete Project Report

From natural-language query to verified, evidence-backed business leads

Repository: github.com/afraziaumer/AI-BDM

Report date: July 2026

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Objectives & Goals.....	4
2.1 Primary Objective	4
2.2 Supporting Goals	4
3. Technology Stack.....	5
3.1 Scraping & Orchestration Layer (Developer A)	5
3.2 RAG Knowledge Layer (Developer B — this report's core focus).....	5
4. System Workflow	6
4.1 Stage-by-Stage Flow	6
Stage 1 — Intent Deconstruction	6
Stage 2 — Discovery	6
Stage 3 — Two-Tier Scraping.....	6
Stage 4 — Classification & Extraction.....	6
Stage 5 — Qualification & Governance	7
Stage 6 — High-Intent Page Selection.....	7
Stage 7 — Knowledge Ingestion & Retrieval (RAG layer).....	7
5. File Structure	7
5.1 Root — Scraping & Orchestration (Developer A)	7
5.2 rag/ — Knowledge & Retrieval Layer (Developer B)	8
5.3 Root-Level Generated Artifacts (not source code).....	9
6. Key Milestones	9
Milestone 1 — Working End-to-End Pipeline.....	9
Milestone 2 — Accurate Per-Query Reporting.....	9
Milestone 3 — Smarter, Contact-First Crawling.....	9
Milestone 4 — Verified, Ranked Contact Extraction.....	9
Milestone 5 — Query Understanding & Business Qualification	9
Milestone 6 — Classification, Routing & Tech Fingerprinting.....	9
Milestone 7 — Production API Layer.....	9
Milestone 8 — The RAG Knowledge Layer (the largest single milestone)	10
Milestone 9 — Consolidation & Publication	10
7. Challenges & How They Were Resolved.....	10
7.1 Scraper-Side Challenges	10
Missing dependency broke fresh installs	10

Reports mixed results across unrelated queries	10
The crawler missed the pages where contact info actually lives	10
Extracted emails were never confirmed real	10
7.2 RAG-Layer Challenges (this session's primary work).....	10
A single dominant business could bury a rare but important match	11
Literal word matching missed different phrasing for the same idea	11
Matching a word was not the same as matching its meaning	11
Fuzzy matching produced false positives from unrelated words.....	11
Two-word concepts (“physical office,” “user login system”) were handled incorrectly.....	11
A business's evidence could exist in the data but never be visible from page text	11
7.3 Publication Challenges.....	11
The RAG layer had never been committed to version control.....	12
The target publish branch had diverged from the working branch	12
8. Known Limitations & Production-Readiness Gaps	12
8.1 Logical / Accuracy Limitations	12
8.2 Technical / Security Gaps (for production hardening)	Error! Bookmark not defined.
8.3 Recommended Priority Order Before Production.....	Error! Bookmark not defined.
9. Validation & Current Status.....	12
10. Appendix — How to Run	13
Full scrape + governance pipeline	13
Ingest results into the knowledge layer and get the answer	13
Run the RAG regression suite	13
Start the production REST API	13

1. Executive Summary

AI-BDM turns a single plain-English sentence — for example “give me 5 gyms in Lahore with no mobile app” — into a verified list of real business leads: contact emails, phone numbers, physical addresses, and a direct, evidence-backed answer to whatever qualifying condition the user asked about. The system is built in two layers that were developed by two collaborating engineers: a scraping and discovery engine (“Developer A”) that finds, crawls, and qualifies businesses matching a query, and a retrieval knowledge layer (“Developer B”, this report's primary focus) that reads everything the scraper collected and answers the specific yes/no question buried inside the query — without ever guessing or hallucinating, and without calling an LLM to generate the answer.

The project began as a straightforward scraper for contact discovery and grew, over several development cycles, into a full pipeline: query understanding, business discovery, two-tier web scraping, contact extraction with real deliverability verification, business classification, high-intent page selection, technology fingerprinting, and finally a hybrid semantic + keyword retrieval engine that can tell a business-development manager, with evidence, whether a prospect actually has (or lacks) the specific feature they asked about.

This report documents the project end to end: its objectives, the technology stack, the full file structure, the system's workflow, the major milestones reached, the real technical challenges encountered and how each was resolved, and an honest account of the system's current limitations and production-readiness gaps.

2. Objectives & Goals

2.1 Primary Objective

Replace the manual, hours-long process of researching business leads — finding companies in an industry/location, checking whether they have or lack a specific feature (an app, a CRM, a physical office, a login system, and so on) — with a single natural-language query that returns a clean, verified, evidence-backed shortlist.

2.2 Supporting Goals

- Understand an unstructured, plain-English query well enough to derive industry, location, business-count, and one or more qualifying conditions (often phrased as “with no X” or “with X”).
- Discover real candidate businesses for that query — not directories, aggregators, or unrelated results — at a scale of dozens per run.
- Crawl each candidate's website efficiently and respectfully, extracting real, deliverable contact information (not just syntactically valid emails).
- Decide, page by page, which parts of a business's website are actually worth analysing in depth (“high-intent pages”), instead of processing every page equally.
- Answer the query's specific qualifying condition with real evidence from the business's own website — distinguishing “confirmed has it,” “confirmed doesn't have it,” and “no evidence either way” — rather than a blind semantic-similarity guess.
- Do all of the above without an LLM “making up” the final answer — the retrieval layer's output is always a real excerpt of real page text, or a real, already-extracted structured fact (like a street address), never a generated sentence.

- Keep the system's own reasoning honest and auditable: every match shows exactly which words/phrases triggered it, and why.

3. Technology Stack

The two layers of the system are built with deliberately different, purpose-fit toolsets.

3.1 Scraping & Orchestration Layer (Developer A)

Layer	Technology	Purpose
Language / runtime	Python 3, asyncio	Fully asynchronous crawl orchestration for concurrency
HTTP client	aiohttp	Native (Tier-1) page fetching
Scraping proxy	Scrape.do API / Zenrows	Tier-2 fallback: JS rendering + residential proxy for blocked/hard sites
Search / discovery	Serper API	Google web + Places search results
LLM provider	Groq (openai/gpt-oss-20b, fallback openai/gpt-oss-120b)	Query-intent parsing, business classification, route planning
HTML parsing	BeautifulSoup4, lxml	DOM parsing, content cleaning, contact extraction
Tech fingerprinting	Wappalyzer (python-Wappalyzer)	Detects a business's CMS, analytics, e-commerce, etc.
Contact verification	dnspython (MX/A record lookups)	Confirms an email's domain can actually receive mail
Phone parsing	phonenumbers	E.164 phone number normalization
Domain parsing	tlldextract	Registered-domain extraction (subdomain-safe deduplication)
API layer	FastAPI, Pydantic, Uvicorn	Production-facing REST endpoints
Storage (current)	Local filesystem + CSV/JSON	Per-business text/metadata; designed to migrate to Cloudflare R2 / MongoDB Atlas

3.2 RAG Knowledge Layer (Developer B — this report's core focus)

Layer	Technology	Purpose
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Turns text into 384-dimension meaning vectors
Vector database	ChromaDB (local, persistent)	Stores and queries chunk embeddings

Chunking	Custom recursive splitter (rag/chunker.py)	Paragraph-safe ~220-character chunks with overlap
Typo correction	pyspellchecker	Normalizes misspelled words before fuzzy comparison
Scoring engine	Custom Python (rag/top_matches.py)	Hybrid semantic + IDF-keyword + focus-word scoring
Answer generation	None (by design)	Pure retrieval — no LLM writes the final answer
Optional LLM path	Groq (rag/generator.py, currently unused)	Retained for a possible future summarization step
Testing	Custom regression suite (rag/eval.py)	7 synthetic, fixed-corpus test cases

4. System Workflow

The pipeline runs as one continuous flow from a single command, though each stage is independently re-runnable.

4.1 Stage-by-Stage Flow

Stage 1 — Intent Deconstruction

The user's plain-English query is sent to an LLM (Groq), which returns a structured plan: industry/search terms, location, requested result count, include/exclude keywords, and — for a query naming a specific company — the target domain directly. A precision guard in the prompt design prevents the model from picking overly generic keywords that would cause false matches later (e.g. the word “app” matching inside “happy”).

Stage 2 — Discovery

The structured plan drives a paginated Google search (via Serper) for candidate businesses. Every raw result is triaged — official business site, a directory/aggregator worth mining for more links, or pure noise — before any crawling begins.

Stage 3 — Two-Tier Scraping

Each candidate site is fetched with a plain, low-cost HTTP request first. Only if that is blocked (403/429/503, a WAF signature, or a connection failure) does the system escalate to a paid proxy service capable of rendering JavaScript and rotating IPs — keeping cost proportional to difficulty rather than paying a premium for every single site. The crawler favours contact-relevant pages (sitemap entries, known contact-page paths, multilingual equivalents) before general link-following, since that is where the highest-value information usually lives.

Stage 4 — Classification & Extraction

While a site is being crawled, several things happen in parallel: a classifier decides whether the site is a genuine business (versus a directory or unrelated page); a technology fingerprinter identifies the site's

CMS/analytics/e-commerce stack; and a multi-technique contact extractor pulls emails, phone numbers, and physical addresses — including from obfuscated formats (Cloudflare-encoded emails, base64-encoded text, schema.org structured data). Extracted emails are verified for real deliverability via a live DNS MX-record lookup, not just checked for correct syntax.

Stage 5 — Qualification & Governance

Each business is checked against the query's include/exclude keywords to confirm it genuinely matches what was asked for. A separate governance pass then cleans the whole dataset: dropping incomplete records, deduplicating by registered domain, and producing a final clean export alongside a quarantine file listing everything that was rejected and why.

Stage 6 — High-Intent Page Selection

For each qualifying business, an LLM route planner reviews lightweight previews of every crawled page (titles, headings, first/last lines — never the full page text) and selects the 3–8 pages most likely to describe what the business actually offers. Only these pages are handed to the knowledge layer — a 40-page site contributes a handful of pages, not all forty.

Stage 7 — Knowledge Ingestion & Retrieval (RAG layer)

Each selected page is split into small, paragraph-safe text chunks and converted into meaning vectors, stored in a local vector database. When a query is answered, every stored chunk is scored by a hybrid formula (semantic closeness + rare-keyword bonus + “focus concept” closeness), ranked into a single combined list across all businesses, and then aggregated per business into a plain verdict: confirmed present, confirmed absent, mixed signals, or no evidence found — the actual deliverable a business-development manager reads.

5. File Structure

5.1 Root — Scraping & Orchestration (Developer A)

File	Approx. size	Responsibility
main.py	6 KB	CLI entry point; orchestrates the full run end to end
phase1_pipeline.py	136 KB	The core crawler: discovery, two-tier scraping, contact extraction, staging
storage.py	20 KB	Filesystem persistence abstraction (staging → commit lifecycle)
route_planner.py	31 KB	LLM selection of each business's high-intent pages
LLM_planner.py	16 KB	Shared Groq client; query-intent parsing and business classification prompts
crawl_planner.py	24 KB	Optional LLM crawl-budget planner (currently disabled by a feature flag)
website_classifier.py	29 KB	Real business vs. directory vs. irrelevant page classification

discovery_classifier.py	22 KB	Pre-crawl triage of raw search results
route_filter.py	41 KB	Heuristic + LLM scoring for page selection
email_extractor.py	33 KB	Multi-technique email discovery, MX verification, ranking
linkedin_finder.py	11 KB	LinkedIn company-page discovery via search (not direct scraping)
tech_stack.py	36 KB	Wappalyzer-based technology fingerprinting
data_pipeline.py	26 KB	Final governance/cleaning stage → leads_clean.csv / .json
api.py	6 KB	FastAPI production wrapper (health, run, list, count endpoints)
requirements.txt	—	Scraper-side Python dependencies

5.2 rag/ — Knowledge & Retrieval Layer (Developer B)

File	Approx. size	Responsibility
config.py	small	Chunk size, embedding model name, Chroma paths, tunable constants
chunker.py	small	Paragraph-safe recursive text splitter
embedder.py	small	Loads and wraps the sentence-embedding model
store.py / pipeline.py	small	Vector-database wiring and the shared ingest pipeline object
contract.py	small	The SourceDoc data contract shared across ingestion paths
ingest_from_storage.py	medium	Production path: reads the scraper's high-intent pages into RAG format
top_matches.py	large (~900 lines)	The scoring engine — hybrid ranking, fuzzy matching, negation, phrase gating, evidence classification
ingest_and_answer.py	medium	The one command run after a scrape — prints ranked results + per-business verdicts
eval.py	medium	Permanent regression test suite — 7 fixed synthetic cases
ingest_from_csv.py, match_query.py, llm_match_query.py, generator.py, ask.py	small each	Earlier/alternate query paths, largely superseded by the current pure-retrieval design
requirements-rag.txt	—	RAG-layer Python dependencies

5.3 Root-Level Generated Artifacts (not source code)

crawl_index.csv (master per-page metadata index), leads_clean.csv / leads_clean.json (final governed lead list), leads_quarantine.csv (rejected records with reasons), data_quality_report.txt, last_run.json, site_classifications.json, storage/<domain>/ (per-business raw crawl output), and rag/.chroma/ (the local vector database). All of these are regenerated by running the pipeline and are excluded from version control.

6. Key Milestones

The project progressed through clearly defined phases, each closing a specific gap found through real usage rather than being planned upfront in full.

Milestone 1 — Working End-to-End Pipeline

Repository cloned, dependencies installed, API keys (Groq, Serper, Scrape.do) configured, and the first successful full run completed — confirming the basic query → discovery → scrape → export loop worked.

Milestone 2 — Accurate Per-Query Reporting

Fixed a data-quality report that mixed results from every query ever run together, by introducing last_run.json to scope each report to the query that actually produced it.

Milestone 3 — Smarter, Contact-First Crawling

Rebuilt the crawler to read robots.txt and XML sitemaps, probe sixteen known contact-page paths across seven languages, and prioritise these high-value pages ahead of generic link-following — closing the single biggest source of missed contact information.

Milestone 4 — Verified, Ranked Contact Extraction

Added live DNS MX-record verification (never sending mail) so extracted emails are confirmed deliverable, not just syntactically valid; disposable-domain rejection; and a ranking system that favours role accounts (info@, sales@) and personal names over generic or clearly-fake addresses.

Milestone 5 — Query Understanding & Business Qualification

Built the LLM-driven intent parser with a keyword-expansion rule and a precision guard, and the lead-qualification stage that checks each scraped business against the query's include/exclude terms — with an explicit “no_content” outcome rather than silently passing empty results.

Milestone 6 — Classification, Routing & Tech Fingerprinting

Added the website classifier (real business vs. directory vs. junk, language-aware), the discovery-result triage stage, the LLM route planner that selects each business's high-intent pages, and Wappalyzer-based technology detection.

Milestone 7 — Production API Layer

Wrapped the pipeline in a FastAPI service with health, run, list, and count endpoints, gated by a static API-key header, ready for programmatic/external use.

Milestone 8 — The RAG Knowledge Layer (the largest single milestone)

Designed and built the entire retrieval system from first principles: chunking, embedding, hybrid semantic + keyword scoring, fuzzy word matching with typo correction, negation and soft-negation awareness, phrase-adjacency and proximity gating, automatic focus-concept detection, and a per-business evidence summary corroborated across pages and cross-checked against structured contact data. Backed throughout by a permanent 7-case regression suite and extensive real-world validation across marina, gym, dental, driving-school, and pet-grooming datasets.

Milestone 9 — Consolidation & Publication

Reviewed and committed the entire RAG layer (previously never version-controlled) alongside a JSON export addition to the governance pipeline, resolved a diverged remote branch, and published the combined result to the project's shared GitHub branch.

7. Challenges & How They Were Resolved

The list below captures genuine problems discovered through real data — not hypothetical risks — each with the fix actually shipped and, where relevant, its current status.

7.1 Scraper-Side Challenges

Missing dependency broke fresh installs

The phonenumbers package was used but absent from requirements.txt, breaking clean installs. Installed and flagged for the requirements file.

Reports mixed results across unrelated queries

Because the raw scrape cache is cumulative, a fresh query's data-quality report silently included businesses from every prior run. Resolved by scoping reports to the latest run via last_run.json, with an explicit --all flag for the cumulative view when genuinely needed.

The crawler missed the pages where contact info actually lives

Following only homepage links one level deep meant the crawler frequently reached a site without ever finding its /contact or /about page. Resolved with sitemap/robots-aware discovery and direct probing of known contact-page paths across multiple languages, fetched ahead of general link-following.

Extracted emails were never confirmed real

Every email was syntax-valid only, with no guarantee the domain could actually receive mail. Resolved with live MX-record verification (DNS lookup only, no mail ever sent), disposable-domain rejection, and per-domain caching so a twelve-page site costs one DNS lookup, not twelve.

7.2 RAG-Layer Challenges (this session's primary work)

A single dominant business could bury a rare but important match

Early scoring let several “somewhat common” word matches (e.g. a shared city name) outweigh one genuinely rare, distinguishing word. Resolved with IDF (inverse document frequency) weighting — a word's bonus scales inversely with how many of the current candidate businesses already use it — combined with diminishing-returns (noisy-OR) stacking so no pile of common words can out-rank one truly distinctive one.

Literal word matching missed different phrasing for the same idea

A page saying “digital booking portal” would not match a search for “app,” despite meaning nearly the same thing. Resolved with embedding-based fuzzy matching (no hardcoded synonym list) plus a dedicated “focus word” mechanism that scores every chunk by closeness to the query's single most important concept, not just literal word overlap.

Matching a word was not the same as matching its meaning

“we just launched our app” and “we have no app” both contain the word “app” but mean opposite things. Resolved with a negation-detection heuristic that checks for cues like “no”/“not” before a matched word, and softer cues like “coming soon”/“in progress” after it, penalizing polarity mismatches rather than treating any word match as a positive signal.

Fuzzy matching produced false positives from unrelated words

The single most persistent problem this session: words whose AI-generated “meaning” happened to sit close to a target word by coincidence, not by actual relation. Concretely: “phone” was mistaken for “app” (fixed by raising the similarity threshold); a website typo, “appointment,” was mistaken for “app” (fixed with pre-comparison spell-correction); and “mobile” / “programs” were mistaken for “app” / “apps” with no clean threshold able to separate them from genuinely related words like “appstore.” This last case remains an open, documented limitation — see Section 8.

Two-word concepts (“physical office,” “user login system”) were handled incorrectly

Three separate, compounding bugs were found and fixed here. First, two unrelated words merely sharing a short text chunk (“a physical certificate” and “our office” in the same paragraph) were wrongly read as proof of a “physical office” — fixed by requiring phrase-pair words to actually sit near each other in the text, not just anywhere in the same chunk. Second, a business repeatedly and genuinely using a word like “office” across several different pages was initially given no credit at all once the first fix was strict — resolved by allowing repeated, cross-page usage of a word to count as real evidence on its own. Third, for a query whose entire real ask was a multi-word phrase (“no user login system”), the system was picking an unrelated filler word (“Miami,” the location context) as its main focus purely because every genuinely relevant word had been excluded by the phrase-detection logic — fixed by preferring explicitly negated words (words directly following “no”) when choosing the query's focus concept.

A business's evidence could exist in the data but never be visible from page text

A driving school's real street address existed in its structured contact data but was never phrased using words like “physical” or “office” anywhere on its site, so no amount of text-matching could ever surface it. Resolved by teaching the evidence system to also check the scraper's already-extracted address field directly for location-related queries — treating a real address as authoritative structured proof, independent of page wording.

7.3 Publication Challenges

The RAG layer had never been committed to version control

The entire knowledge-retrieval system, built over an extended development effort, existed only in the local working directory. Reviewed for accidental secrets or generated cache artifacts (none found, beyond expected local vector-store binaries which were excluded), and committed as a single, complete addition.

The target publish branch had diverged from the working branch

The shared branch intended for this work had three of its own commits never merged into the main line, while the main line itself was seventeen commits ahead — a genuine, hard-to-reverse divergence. Rather than silently resolving this, the situation was surfaced explicitly and the resolution strategy (force-publishing the current, complete state) was confirmed before acting, since it would permanently remove those three commits from that branch.

8. Known Limitations & Production-Readiness Gaps

An honest system knows what it doesn't know. The following are documented, real gaps — not hypothetical risks — identified through direct testing and code review of the finished system.

8.1 Logical / Accuracy Limitations

- Fuzzy-matching false positives with no clean fix: “mobile” and “programs” can still be mistaken for “app”/“apps” — measured similarity scores place the false matches too close to genuinely related words (like “appstore”) to separate with a simple threshold.
- “No evidence found” is not the same as “doesn't have it” — it only means the business's website never mentioned the topic either way; this is communicated clearly in the output, but remains a hard ceiling on what a website-only analysis can ever know.
- Multi-word compound queries rely on heuristics that needed three successive rounds of fixes this session alone and may still have untested phrasings that trigger similar edge cases.
- There is no large, labeled ground-truth dataset to compute a formal precision/recall figure against; confidence instead comes from a small permanent regression suite (7 cases, currently all passing) plus extensive manual spot-checking across six different real business categories this session.

9. Validation & Current Status

The RAG layer's regression suite currently passes all 7 of its fixed test cases, covering diminishing-returns keyword scoring, cross-page boilerplate stripping, embedding-based fuzzy matching, hard negation, soft negation with varied phrasing, and context-based understanding without literal word overlap. Beyond synthetic testing, the system was validated live across marina businesses, driving schools, gyms, dental clinics, and pet-grooming salons, with every discrepancy investigated to completion — either confirmed as correct-but-surprising behaviour, or diagnosed to a real bug and fixed, then re-verified against both the regression suite and the original real-world case that surfaced it.

As of this report, the complete codebase — the scraper pipeline and the full RAG knowledge layer — is committed to version control and published to the project's shared repository, ready for the next phase of development.

10. Appendix — How to Run

Full scrape + governance pipeline

```
python main.py --query "5 gyms in Lahore with no mobile app"
```

Ingest results into the knowledge layer and get the answer

```
python -m rag.ingest_and_answer
```

Run the RAG regression suite

```
python -m rag.eval
```

Start the production REST API

```
python -m uvicorn api:app --reload --port 8000
```

Outputs after a full run: leads_clean.csv / leads_clean.json (final governed leads), leads_quarantine.csv (rejected records with reasons), data_quality_report.txt, and — after the RAG step — a per-business evidence verdict for the query's qualifying condition, printed directly to the terminal.
